

How the web app fits together

A picture-first guide to the data pipeline, API, dashboard, and Groq-backed resume fit flow.

515

companies in the local dataset

24,898

active job rows

68,651

job-skill links

2

cached resume profiles

Framework stack

Next.js 16

React 19

TypeScript

Tailwind v4

Recharts

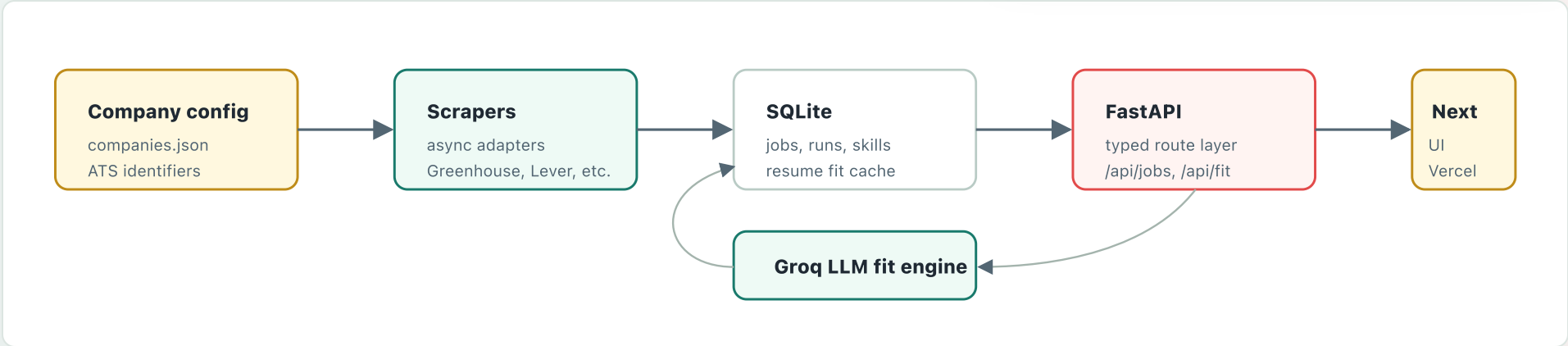
FastAPI

SQLite

Groq API

Playwright

Frontend and backend deploy separately: Vercel serves `dashboard/`, Railway runs the FastAPI app from the repo root.



The tracker starts as a batch pipeline

1. Load tracked companies

name, ATS type, identifier, sector, company_type

2. Run async scraper adapter

Greenhouse, Lever, Ashby, SmartRecruiters, Teamtailor
plus generic static / Playwright fallbacks

3. Sync against previous active jobs

insert new postings, mark missing postings inactive

4. Enrich fields for the dashboard

seniority, department, work model, skills

Orchestrator shape

```
async def run():
    init_db()
    companies = load_companies()
    company_ids = {c["name"]: upsert_company(c) for c in companies}

    semaphore = asyncio.Semaphore(SCRAPE_CONCURRENCY)
    tasks = [
        scrape_company_limited(semaphore, c,
                               company_ids[c["name"]])
        for c in companies
    ]
    results = await asyncio.gather(*tasks,
                                   return_exceptions=True)
```

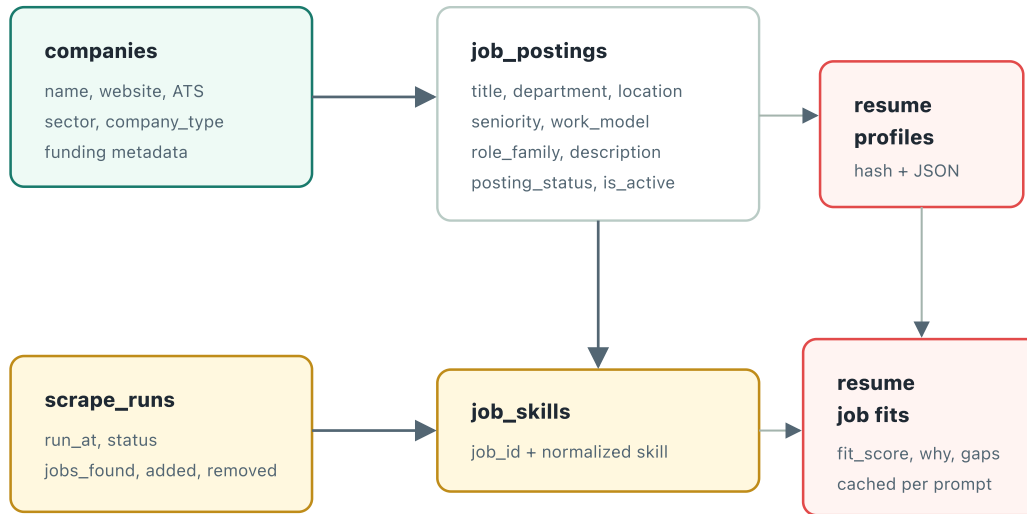
Why this shape works

Scraping is I/O bound, so concurrency gives speed without complicating the database write path. Each run records found, added, removed, and failed counts, which powers health and change views.

Where the app learns structure

Raw postings become dashboard-friendly fields after enrichment: normalized department, seniority, work model, role family, and skill tags.

SQLite is the product memory



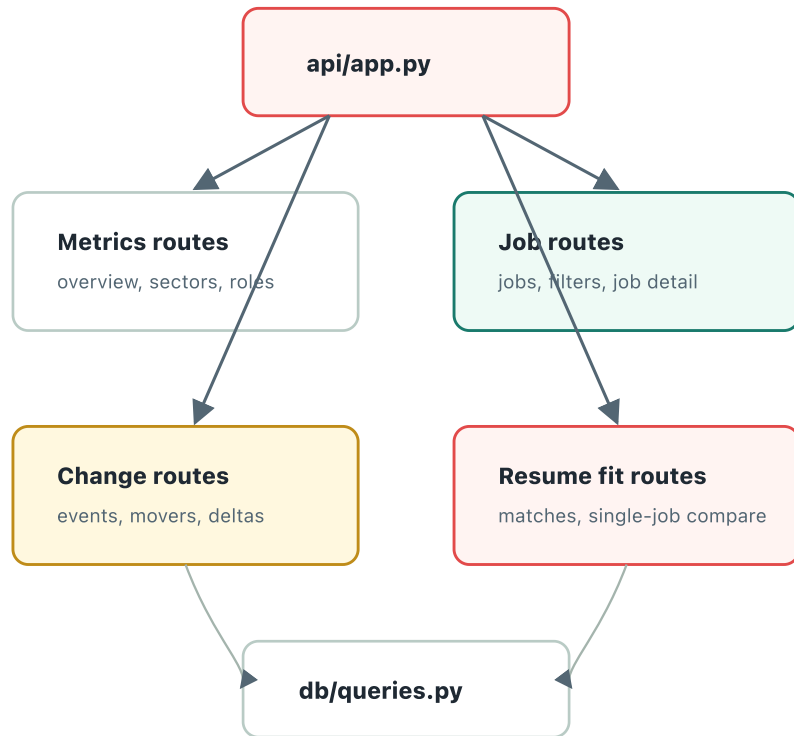
Schema source

```
CREATE TABLE IF NOT EXISTS resume_job_fits (
  resume_id INTEGER NOT NULL,
  job_id INTEGER NOT NULL,
  deterministic_score REAL NOT NULL,
  fit_score INTEGER NOT NULL,
  rationale_json TEXT NOT NULL,
  gaps_json TEXT NOT NULL,
  suggestions_json TEXT NOT NULL,
  prompt_version TEXT NOT NULL,
  UNIQUE(resume_id, job_id, prompt_version)
);
```

Constraint that matters

Raw resume text is not stored. The DB stores a resume hash, a compact extracted profile JSON, and per-job fit outputs. That makes repeat comparisons cheaper while keeping the sensitive raw upload out of persistence.

FastAPI is a thin route layer over query helpers



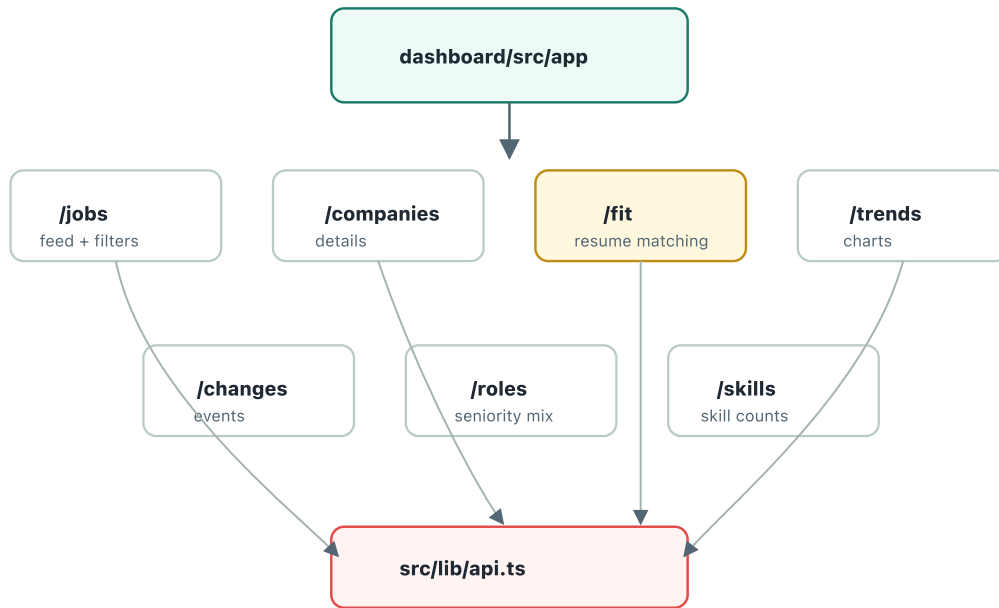
Example job endpoint

```
@app.get("/api/jobs")
def jobs(search: Optional[str] = None,
         company_type: Optional[str] = None,
         seniority: Optional[str] = None):
    return queries.get_active_jobs(
        search=search,
        company_type=company_type,
        seniority=seniority,
    )
```

Example resume endpoint

```
@app.post("/api/fit/matches")
async def fit_matches(resume: UploadFile = File(...),
                     company_type: Optional[str] = Form(None),
                     limit: int = Form(20)):
    content = await resume.read()
    resume_text = extract_resume_text(resume.filename, content)
    return analyze_resume_matches(
        resume_text,
        company_type=company_type or None,
        limit=max(1, min(limit, 40)),
    )
```

Next.js pages read typed API helpers



Job Feed screen

Title	Company	Work	Fit
			Compare
			Compare
			Compare

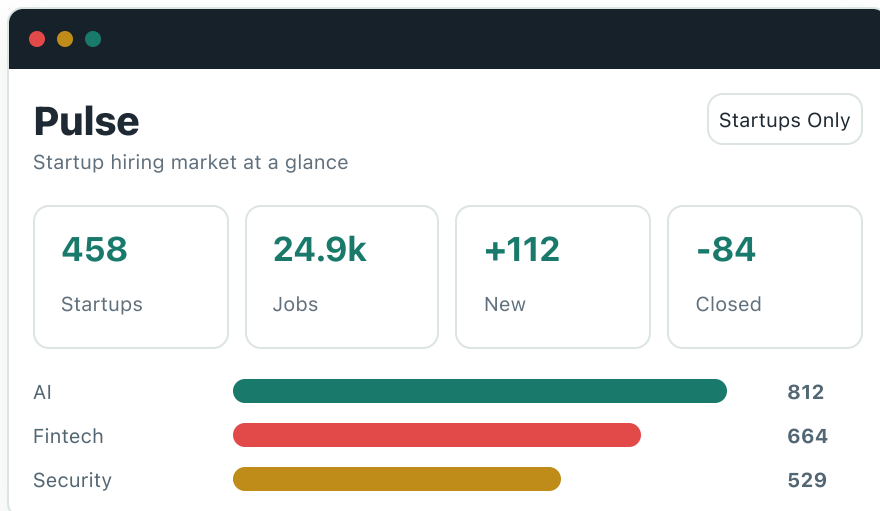
API client pattern

```
const API_BASE =
  process.env.NEXT_PUBLIC_API_URL ||
  "http://localhost:8000";

jobs: (filters?: JobFilters) =>
  fetchApi<JobRow[]>("/api/jobs", filters)

fitMatches: (file, options) => {
  const form = new FormData();
  form.set("resume", file);
  form.set("company_type",
    options.company_type);
  return postFormApi("/api/fit/matches", form);
}
```

How a dashboard page is assembled



The page has three jobs

- 1 Declare client-side state for data and UI controls: overview rows, chart rows, selected scope, and chart mode.
- 2 Load data with `useEffect` whenever the scope changes. The page does not talk to SQL directly; it calls typed API helpers.
- 3 Render small reusable visual blocks, then feed arrays into Recharts for responsive charts.

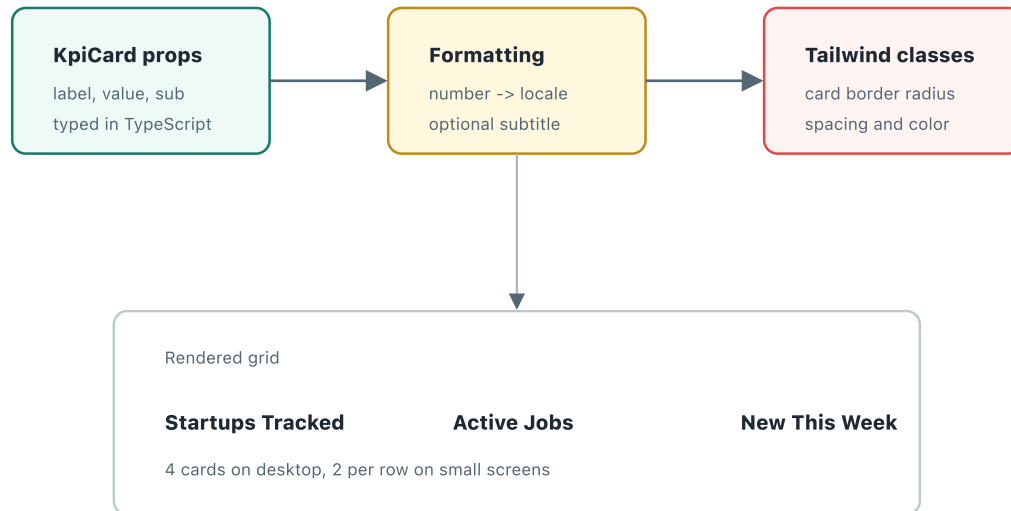
Code skeleton

```
export default function PulsePage() {
  const [overview, setOverview] = useState(null);
  const [scope, setScope] = useState<"startup" | "all">("startup");

  useEffect(() => {
    const ct = scope === "startup" ? "startup" : undefined;
    api.overview(ct).then(setOverview);
    api.departments(ct).then(setDepartments);
    api.roleFamilies(ct).then(setRoleFamilies);
  }, [scope]);

  return <div className="space-y-8">...</div>;
}
```

KPI cards are just small typed components



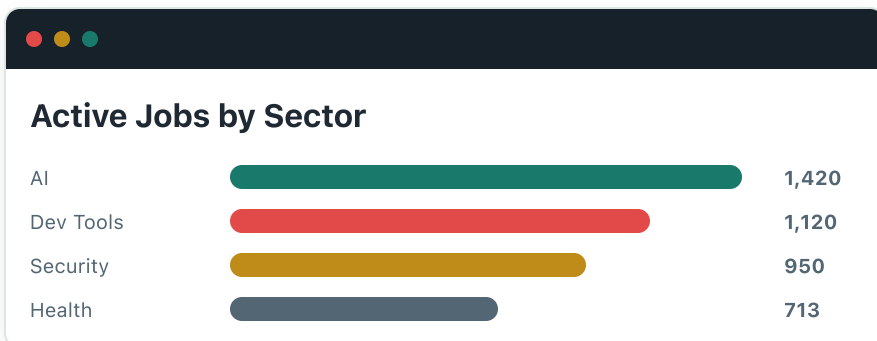
Component implementation

```
function KpiCard({ label, value, sub }: {  
  label: string;  
  value: string | number;  
  sub?: string;  
}) {  
  return (  
    <div className="bg-card border border-card-  
border rounded-xl p-5">  
      <p className="text-sm text-muted">{label}</p>  
      <p className="text-2xl font-bold mt-1">  
        {typeof value === "number" ?  
          value.toLocaleString() : value}</p>  
      {sub && <p className="text-xs text-muted  
mt-1">{sub}</p>}</div>  
  );  
}
```

Tutorial takeaway

The visual design is mostly composition: a typed function, a few props, and utility classes. Once the component exists, the page can render a consistent row of metrics without repeating markup.

Recharts turns API rows into responsive visuals



- 1 The API returns plain rows like `{ sector: "AI", job\_count: 1420 }`.
- 2 `ResponsiveContainer` lets the chart fill the card width without hand-computing pixels.
- 3 `Cell` maps each bar to a color from the palette, so the data array drives the visual.

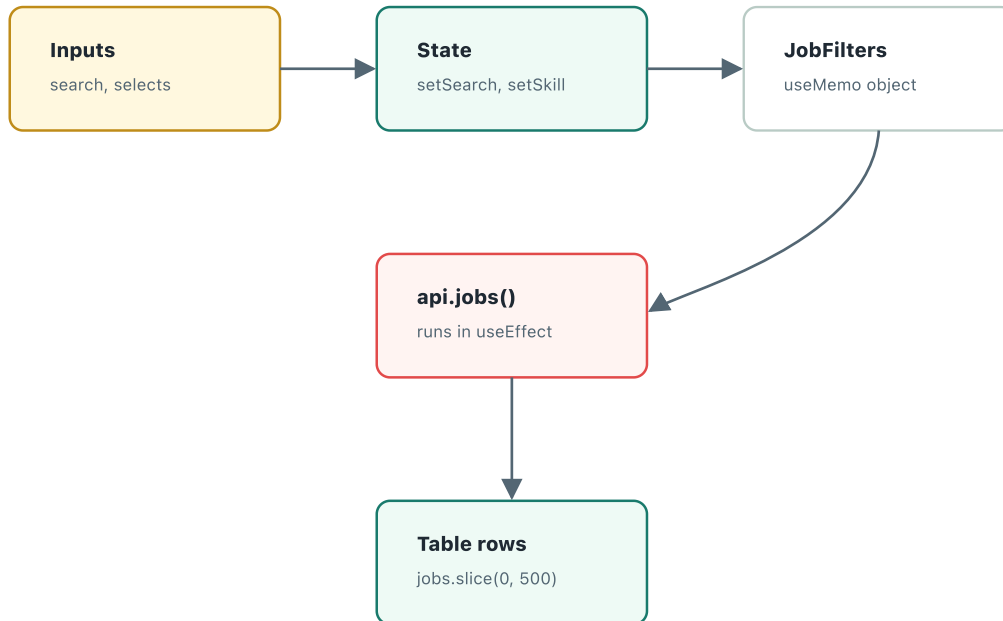
Chart code

```
<ResponsiveContainer width="100%" height={360}>
  <BarChart data={sectors} layout="vertical" margin={{ left: 20
}}>
    <XAxis type="number" tick={{ fill: "#9ca3af", fontSize: 12 }} />
  />
  <YAxis type="category" dataKey="sector" width={160} />
  <Tooltip {...TOOLTIP_STYLE} />
  <Bar dataKey="job_count" radius={[0, 4, 4, 0]}>
    {sectors.map((_, i) => (
      <Cell key={i} fill={COLORS[i % COLORS.length]} />
    ))}
  </Bar>
</BarChart>
</ResponsiveContainer>
```

Why the charts feel integrated

The chart container uses the same card shell, border, radius, text color, and tooltip style as the rest of the app. Recharts handles SVG drawing; the app owns layout, labels, and data shaping.

The filter bar is controlled React state



Controlled input pattern

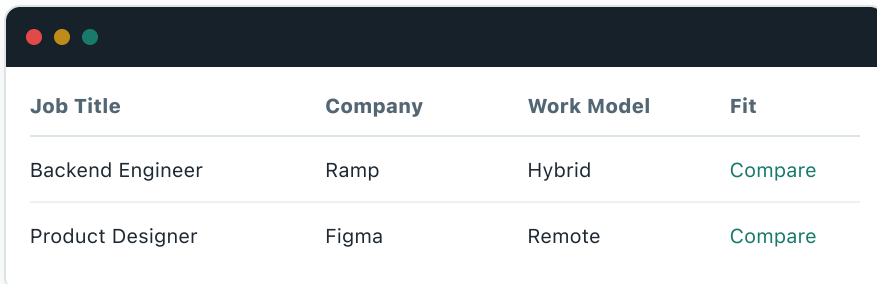
```
<input
  type="text"
  placeholder="Search by job title..."
  value={search}
  onChange={(e) => setSearch(e.target.value)}
  className="bg-background border border-card-
border rounded-lg px-3 py-2"
/>
```

Fetch when filters change

```
const filters = useMemo(() => ({
  search: search || undefined,
  skill: skill || undefined,
  company_type: companyType || undefined,
}), [search, skill, companyType]);

useEffect(() => {
  let cancelled = false;
  api.jobs(filters).then((data) => {
    if (!cancelled) setJobs(data);
  });
  return () => { cancelled = true; };
}, [filters]);
```

The job table is a visual router into resume comparison



Job Title	Company	Work Model	Fit
Backend Engineer	Ramp	Hybrid	Compare
Product Designer	Figma	Remote	Compare

- 1 The table keeps a sticky header and scrollable body so thousands of jobs stay navigable.
- 2 Cell values are formatted before display: role families, seniority, work model, dates, and location fallbacks.
- 3 The `Compare` link encodes the job id in the URL so `/fit` can load selected job context.

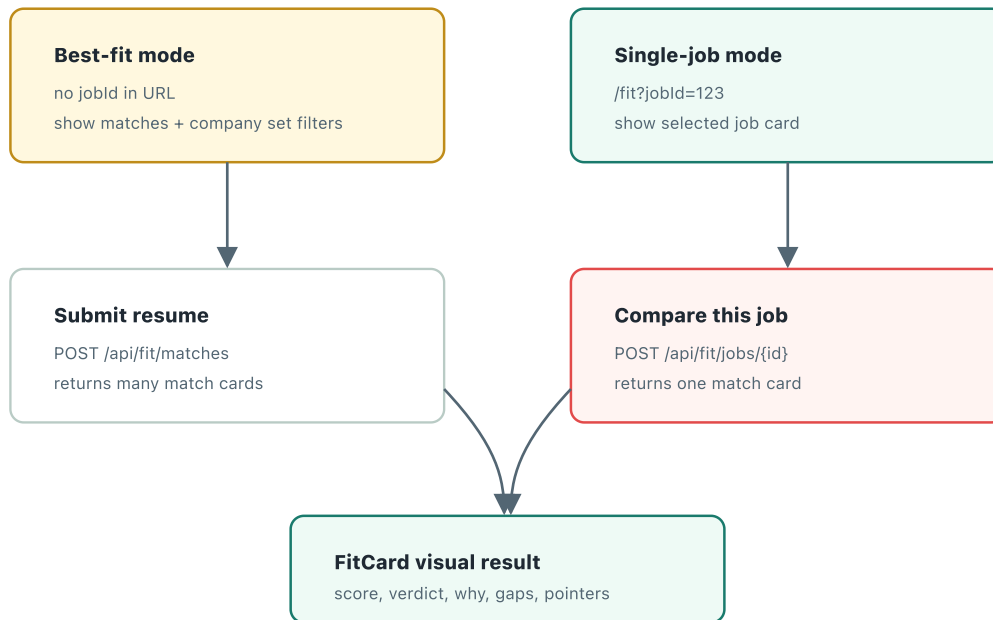
Rows are mapped from API data

```
{jobs.slice(0, 500).map((j) => (  
  <tr key={j.id} className="border-b border-card-border/50">  
    <td className="font-medium max-w-xs truncate">{j.title}</td>  
    <td className="text-muted">{j.company_name}</td>  
    <td className="text-xs">{formatWorkModel(j.work_model)}</td>  
    <td>  
      <Link href={` /fit?jobId=${j.id}`}>Compare</Link>  
    </td>  
  </tr>  
) )}
```

Why links beat hidden state here

A URL like `/fit?jobId=123` is shareable, reloadable, and works from any screen. The `/fit` page reads that query param and fetches the selected job preview.

The fit page has two modes



Query param sync

```
const selectedJobId = params.get("jobId") ||
"";

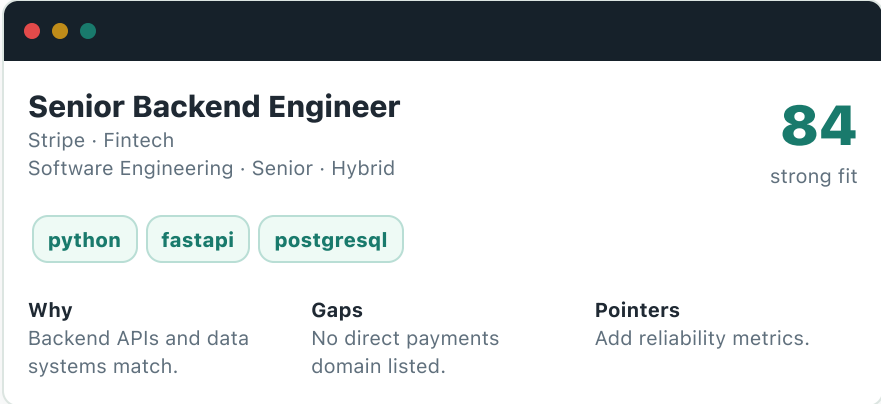
useEffect(() => {
  setJobId(selectedJobId);
  setSelectedJob(null);
  if (!selectedJobId) return;

  api.jobDetail(Number(selectedJobId))
    .then(setSelectedJob)
    .catch(() => setJobError("Selected job was
not found."));
}, [selectedJobId]);
```

Mode switch in JSX

```
{!jobId.trim() && (
  <>
    <input type="number" value={limit} />
    <select value={companyType}>
      <option value="">Both</option>
      <option value="startup">Startups</option>
      <option value="bigco">Big
companies</option>
    </select>
  </>
)}
```

AI output is rendered as structured product UI



Senior Backend Engineer

Stripe · Fintech
Software Engineering · Senior · Hybrid

84
strong fit

python fastapi postgresql

Why Backend APIs and data systems match.	Gaps No direct payments domain listed.	Pointers Add reliability metrics.
--	--	---

Visual score color

```
function scoreColor(score: number) {
  if (score >= 80) return "text-green";
  if (score >= 65) return "text-accent-light";
  if (score >= 50) return "text-yellow-400";
  return "text-red";
}
```

Rendering structured LLM output

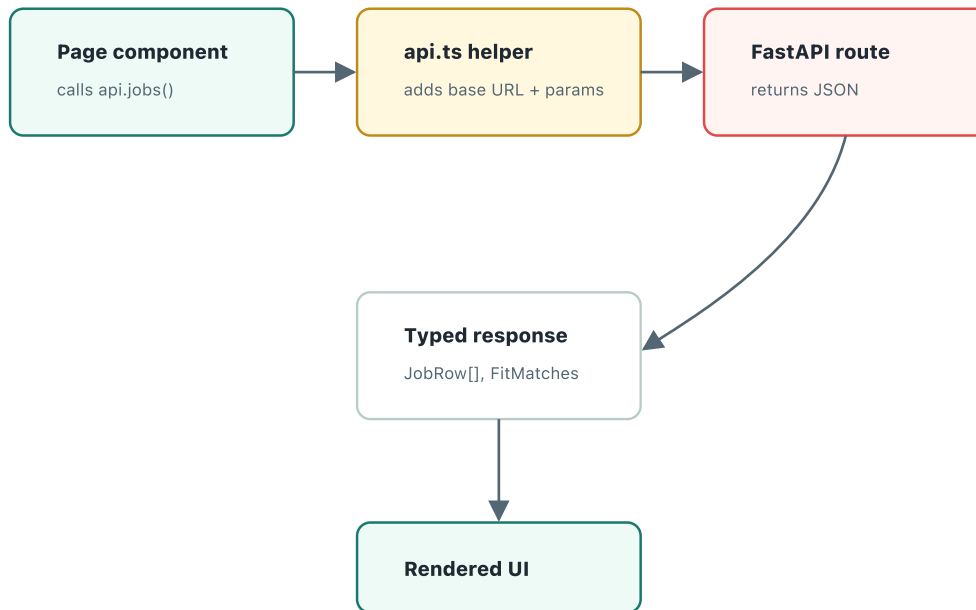
```
<ReasonBlock title="Why" rows={match.why} />
<ReasonBlock title="Gaps" rows={match.gaps} />
<ReasonBlock title="Pointers" rows={match.resume_pointers} />

{match.matched_skills.map((skill) => (
  <span key={skill} className="rounded-full bg-green/10">
    {skill}
  </span>
))}
```

Tutorial takeaway

The LLM does not return HTML. It returns arrays and fields. React decides how to present those fields consistently as score, chips, sections, and links.

The frontend has one gateway to backend data



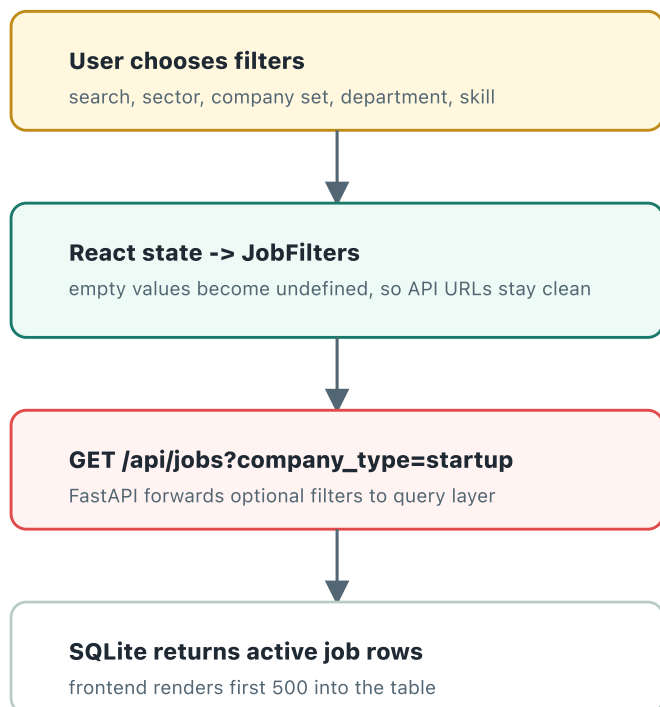
Generic GET helper

```
async function fetchApi<T>(  
  path: string,  
  params?: Record<string, string | number |  
    undefined>  
) : Promise<T> {  
  const url = new URL(`${API_BASE}${path}`);  
  Object.entries(params ?? {}).forEach(([k, v])  
=> {  
    if (v !== undefined && v !== "")  
      url.searchParams.set(k, String(v));  
  });  
  const res = await fetch(url.toString());  
  if (!res.ok) throw new Error(`API error:  
    ${res.status}`);  
  return res.json();  
}
```

Form upload helper

```
fitMatches: (file, options) => {  
  const form = new FormData();  
  form.set("resume", file);  
  if (options?.company_type)  
    form.set("company_type", options.company_type);  
  if (options?.limit) form.set("limit",  
    String(options.limit));  
  return postFormApi<FitMatchesResponse>  
    ("/api/fit/matches", form);  
}
```

Filters become query params, then SQL clauses



Frontend filter object

```
const filters = useMemo(): JobFilters => ({
  search: search || undefined,
  sector: sector || undefined,
  skill: skill || undefined,
  seniority: seniority || undefined,
  work_model: workModel || undefined,
  company_type: companyType || undefined,
  department: department || undefined,
}), [search, sector, skill, seniority, workModel, companyType]);
```

Backend SQL shape

```
where = "jp.is_active = 1"

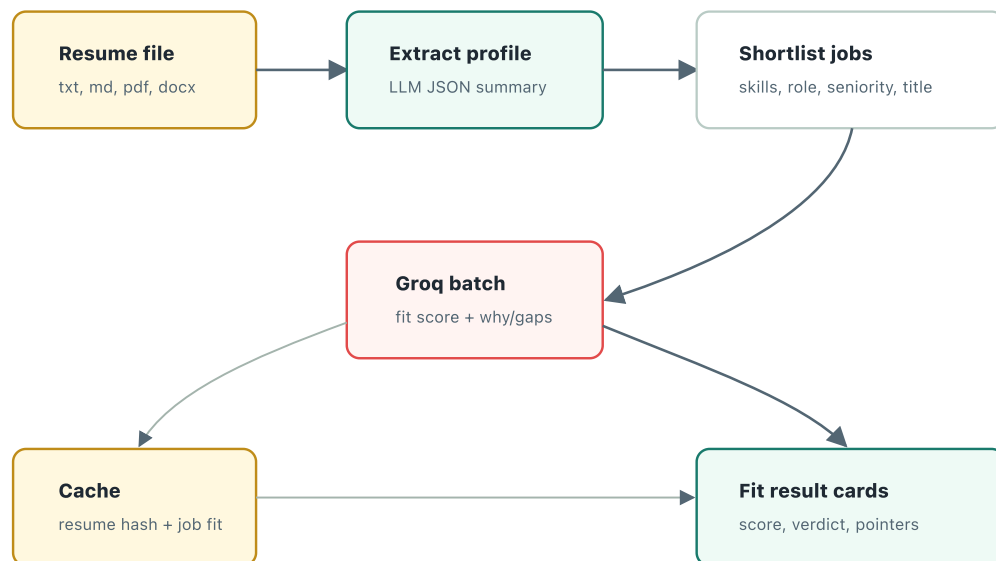
if company_type:
    sql += " AND c.company_type = ?"
    params.append(company_type)

if search:
    sql += " AND (jp.title LIKE ? OR jp.department LIKE ?)"
```

Latest product filter

The job feed and resume-fit search both expose the same choices: Both, Startups, and Big companies. For resume fit, that scope constrains the shortlist before LLM calls happen.

Resume Fit is deterministic shortlist plus Groq reasoning



Shortlist score before LLM

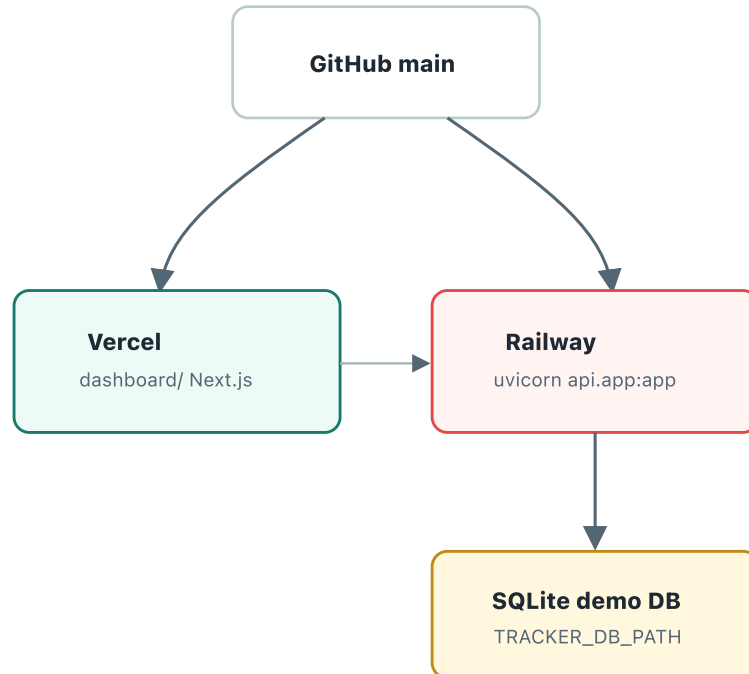
```
score = 0.0
score += min(len(overlap) * 8.0, 40.0)
score += 25.0 if role_family matches else 0.0
score += 15.0 if seniority is compatible else 0.0
score += title_similarity * 12.0
score += 5.0 if sector matches resume domains
else 0.0
score += freshness_score(first_seen_at)
```

Groq call

```
body = {
  "model": _model(),
  "messages": messages,
  "temperature": 0.1,
  "response_format": {"type": "json_object"},
}
```

Location and remote compatibility are context only. They appear in notes or blocker flags, but they are not weighted into fit score.

The deployed app is two services sharing one API contract



Environment contract

```
# Frontend
NEXT_PUBLIC_API_URL=https://your-railway-api

# Backend
TRACKER_DB_PATH=data/tracker-demo.db
FRONTEND_URL=https://your-vercel-app
GROQ_API_KEY=...
LLM_MODEL=...
```

Local development

```
uvicorn api.app:app --reload --port 8000

cd dashboard
npm run dev
```

How to extend it

Add a field in SQLite, expose it in `db/queries.py`, return it from a FastAPI route, type it in `dashboard/src/lib/api.ts`, then render it in a page component. For AI features, keep the raw resume out of storage and cache only structured profiles and fit decisions.

What you would code, in what order

Goal of the manual section

The next pages connect screenshots to the files and functions that create them. Treat it like a guided build: start with layout and theme, add the typed API client, create backend routes, build each screen, then wire resume matching and deployment.

17-18 Screen tour: Pulse and Job Feed

20 Screen tour: Trends and Roles

22 File map and responsibilities

24 Database functions

26 Frontend data client

30 AI fit implementation

19 Screen tour: Resume Fit

21 Screen tour: Companies and Changes

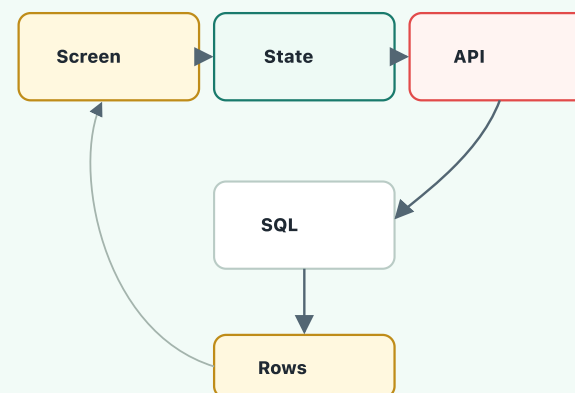
23 Theme, root layout, sidebar

25 API route functions

27-29 Page build recipes

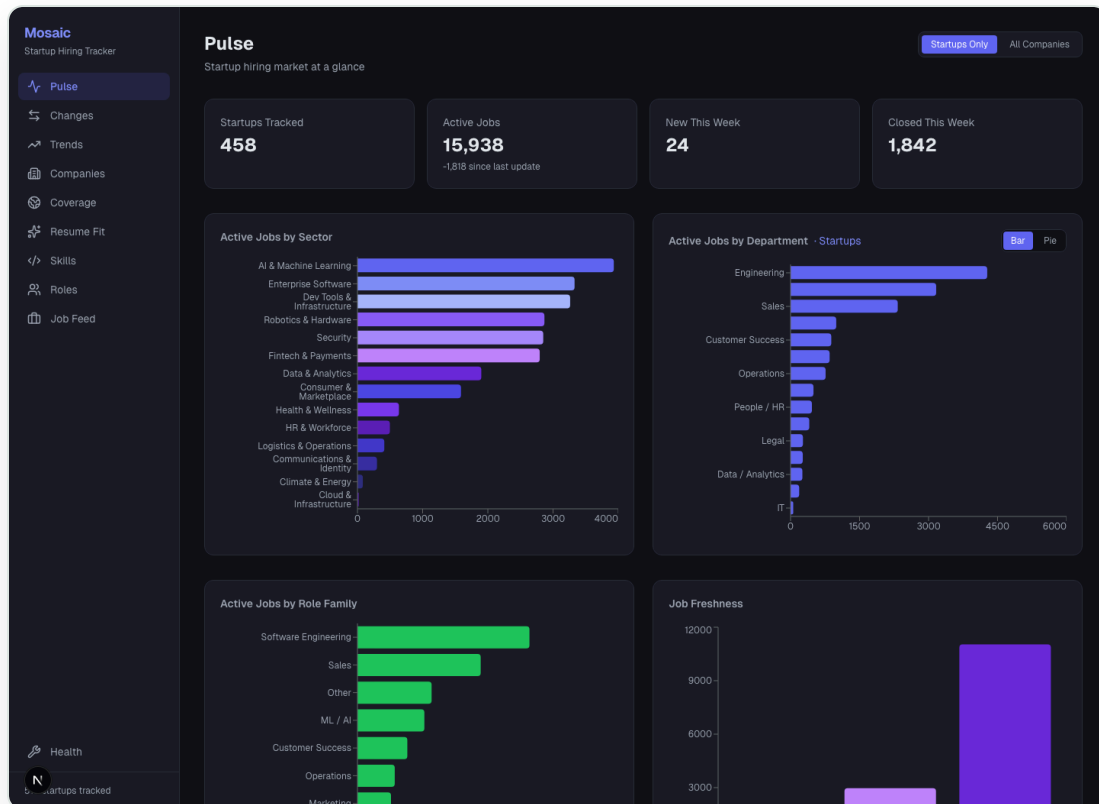
31-33 Recreate checklist

Core mental model



Every visual feature follows the same loop: UI state picks a question, API sends the question to FastAPI, SQL returns rows, and React maps rows into cards, charts, tables, or AI result blocks.

The homepage dashboard is cards plus charts



What you see

- 1 Scope buttons change whether the dashboard asks for startup-only or all-company data.
- 2 KPI cards display totals from `/api/overview`.
- 3 Sector, department, role family, and freshness charts render rows from API helpers.

Files and functions

dashboard/src/app/page.tsx `PulsePage`, `KpiCard`, chart JSX, scope state

dashboard/src/lib/api.ts `api.overview`,
`api.departments`,
`api.roleFamilies`,
`api.jobFreshness`

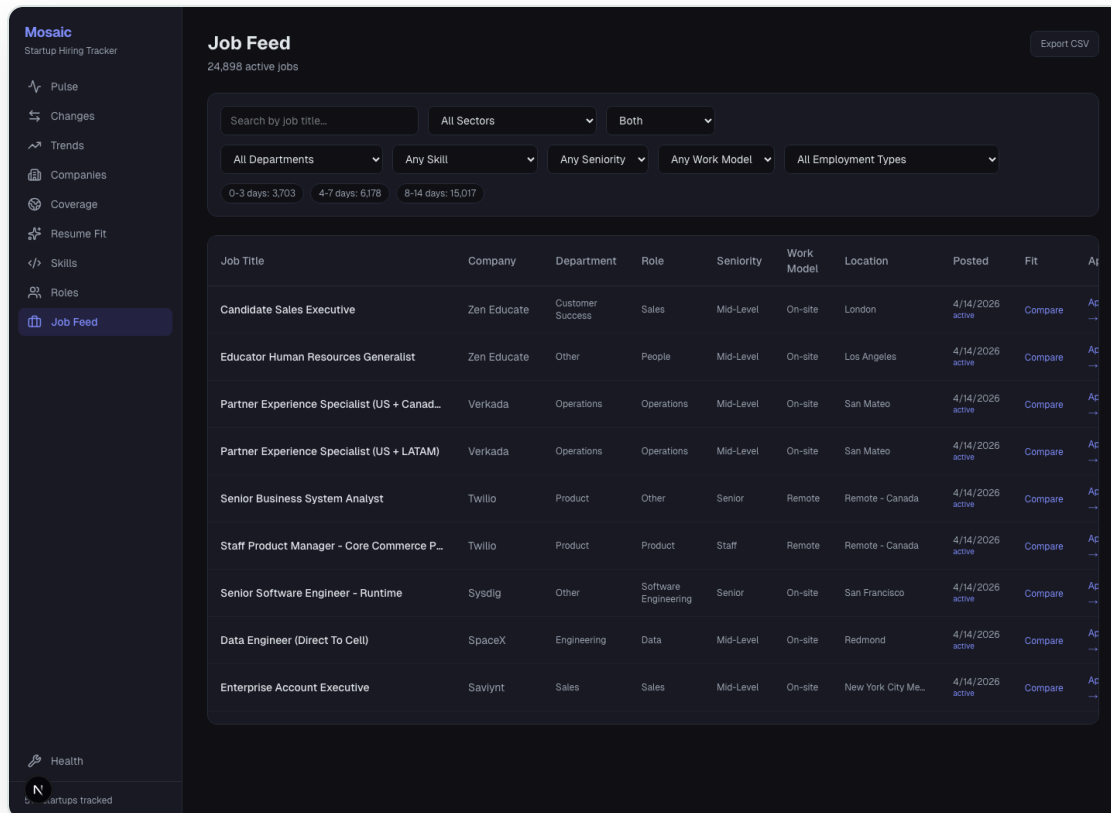
api/app.py `overview`, `departments`,
`role\_families`,
`job\_freshness`

db/queries.py `get\_overview\_stats`,
`get\_department\_breakdown`,
`get\_role\_family\_breakdown`,
`get\_freshness\_breakdown`

Why it affects layout

The dashboard uses a stable grid: `grid-cols-2 md:grid-cols-4` for cards and `md:grid-cols-2` for charts. Data changes the numbers and bar lengths, but the layout stays predictable.

The job feed is a search UI over active job rows



What you see

- 1 Filter controls are controlled inputs: each one stores its current value in React state.
- 2 `useMemo` builds a clean `JobFilters` object, skipping empty fields.
- 3 The table maps `JobRow[]` into visible rows, links company names to detail pages, and routes `Compare` to Resume Fit.

Code recipe

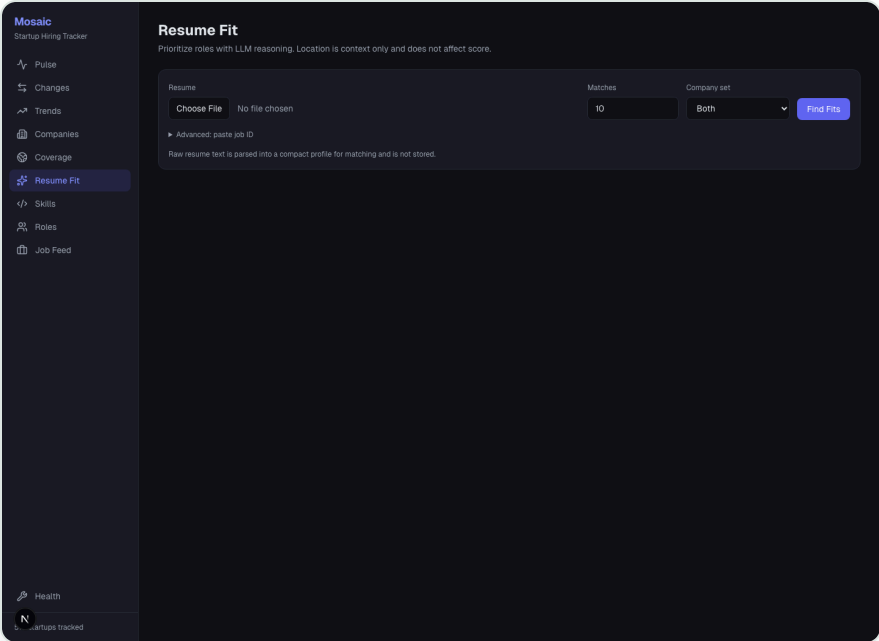
```
const [search, setSearch] = useState("");
const [companyType, setCompanyType] =
  useState("");

const filters = useMemo(() => ({
  search: search || undefined,
  company_type: companyType || undefined,
}), [search, companyType]);

useEffect(() => {
  api.jobs(filters).then(setJobs);
}, [filters]);
```

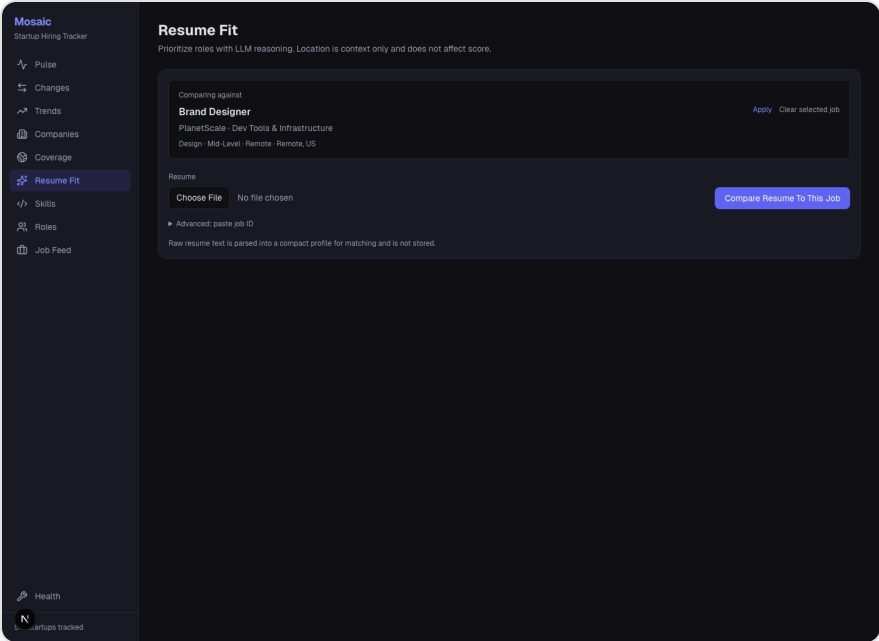
One page, two user journeys

Best-fit mode



No `jobId` in the URL. The form shows match count and company set because the user is asking the app to search broadly.

Single-job mode



With `/fit?jobId=2222`, the page fetches one job and shows the selected-job card above upload controls.

Files and functions

FitInner	owns file, limit, companyType, jobId, selectedJob, loading, error, result state
SelectedJobCard	renders selected job title, company, sector, role, Apply link, clear button
FitCard	renders score, verdict, skills, why, gaps, pointers
ReasonBlock	renders each explanation section from LLM arrays

Mode decision code

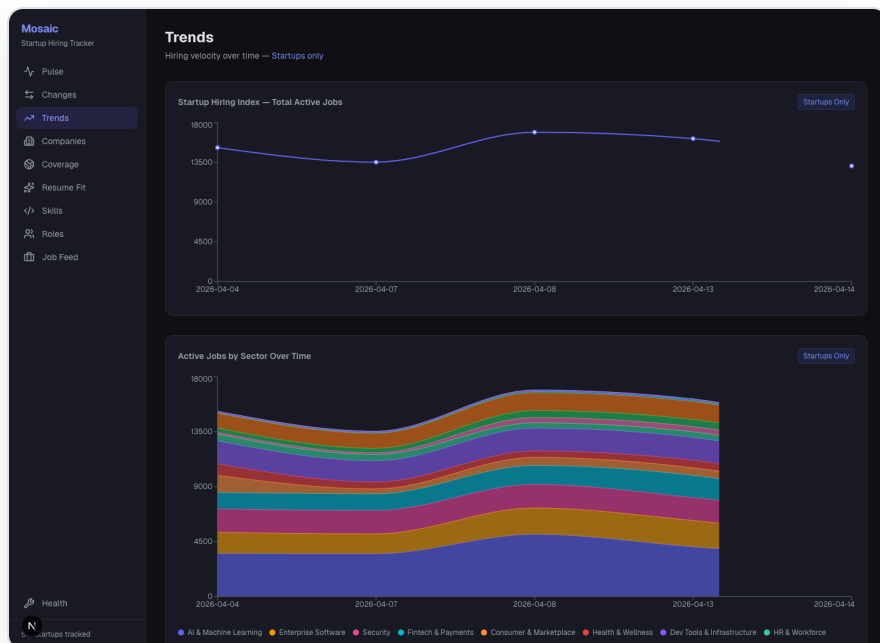
```
const selectedJobId = params.get("jobId") || "";

if (trimmedJobId) {
  const data = await api.fitJob(Number(trimmedJobId), file);
  setResult(data);
} else {
  const data = await api.fitMatches(file, {
    company_type: companyType || undefined,
    limit,
  });
  setResult(data);
}
```

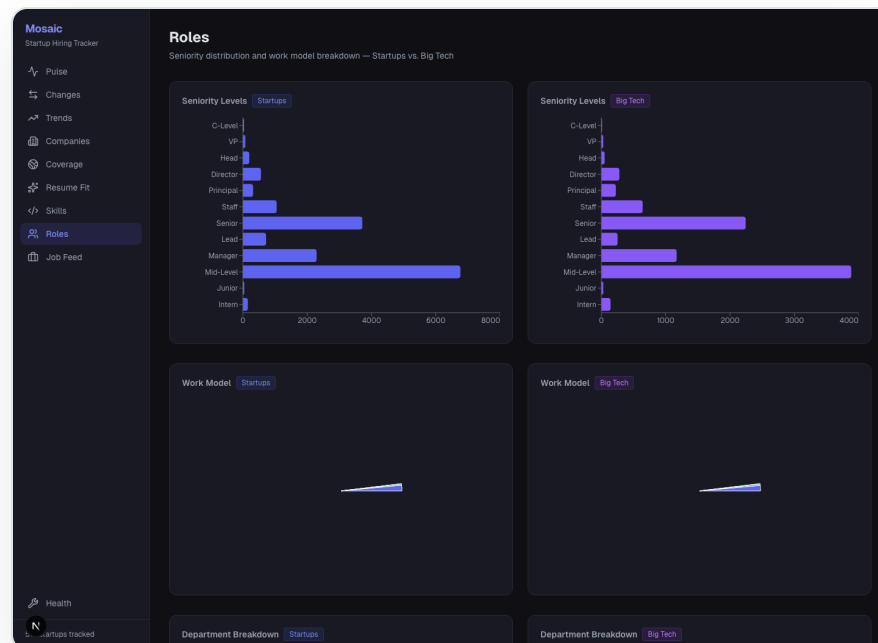
URL query state changes what controls the user sees

More charts reuse the same pattern

Trends



Roles



Line charts

`dashboard/src/app/trends/page.tsx` shapes job count rows into a time series and passes them into `LineChart`.

Stacked bars

Cross-tab rows become multi-series bars by transforming `{ sector, department, job\_count }` into one row per sector.

Pie charts

Roles and company detail pages use `PieChart`, `Pie`, `Cell`, and shared color arrays for category breakdowns.

Tables, badges, and movement views come from query helpers

Companies

Companies
399 of 515 companies

Search companies... All Sectors All Stages Hide 0-job Export CSV

Company	Sector	Funding Stage	Raised	Active Jobs	7-Day Change	Updated
SpaceX	Robotics & Hardware	—	—	1552	+1552	4/14/2026
Databricks	Data & Analytics	—	—	812	-35	4/14/2026
OpenAI	AI & Machine Learning	Series F	\$6600M	635	-3	4/14/2026
Cloudflare	Dev Tools & Infrastructure	—	—	493	-23	4/13/2026
Stripe	Fintech & Payments	—	—	463	-32	4/14/2026
Datadog	Dev Tools & Infrastructure	—	—	439	+6	4/14/2026
Anthropic	AI & Machine Learning	Series E	\$7300M	482	+7	4/13/2026
MongoDB Inc	Data & Analytics	—	—	398	-38	4/14/2026
Samsara	Dev Tools & Infrastructure	—	—	385	+385	4/14/2026
HelloFresh	Consumer & Marketplace	—	—	379	+379	4/14/2026
Okta	Security	—	—	356	-21	4/14/2026
Carva	Enterprise Software	—	—	321	-19	4/13/2026
Zscaler	Security	—	—	305	+305	4/14/2026
Relativity	Security	—	—	298	+298	4/14/2026
Verkada	Security	Series D	\$460M	288	+15	4/14/2026
Shield AI	Security	—	—	249	—	4/14/2026
xAI	AI & Machine Learning	—	—	242	+242	4/14/2026

Health
N startups tracked

Changes

Changes
What changed since the last update

+21 opened -479 closed -458 net

Biggest Changes

SpaceX	Robotics & Hardware	+1552
Samsara	Dev Tools & Infrastructure	+385
HelloFresh	Consumer & Marketplace	+379
Zscaler	Security	+305
Relativity	Security	+298
xAI	AI & Machine Learning	+242
Toast	Fintech & Payments	+239
Sift	Fintech & Payments	+236

Net Change by Sector

Activity Log

Status	Job Title	Company	Sector	Department	Date
Closed	Senior IT Engineer	Waabi	Robotics & Hardware	Other	4/14/2026
Closed	Senior Manager, Procurement Operations	Waabi	Robotics & Hardware	Other	4/14/2026
Opened	Candidate Sales Executive	Zen Educate	HR & Workforce	Customer Success	4/14/2026
Opened	Educator Human Resources Generalist	Zen Educate	HR & Workforce	Other	4/14/2026
Closed	Knowledge Enablement Senior Specialist, AI and Automation	WHOOP	Health & Wellness	Other	4/14/2026
Closed	System Administrator, Doha	WHOOP	Health & Wellness	IT	4/14/2026
Closed	Technical Program Manager (NPi)	WHOOP	Health & Wellness	Operations	4/14/2026
Closed	Marketing Director (ANZ, Temporary)	WHOOP	Health & Wellness	Marketing	4/14/2026

Health
N startups tracked

Companies page

Loads `api.companies()`, then filters/sorts client-side for a management view of tracked companies.

Changes page

Combines event logs, mover rows, and sector deltas so the user can see what changed between scrapes.

Health page

Uses scrape run status to show broken, stale, or healthy company sources.

Every major file has one job

Frontend files

layout.tsx	Wraps every page with fonts, sidebar, and scrollable main content.
globals.css	Defines design tokens: background, card, borders, accent, green, red, muted.
components/Sidebar.tsx	Navigation list, active link styling, mobile drawer, company count.
lib/api.ts	Typed API functions and upload helpers.
lib/format.ts	Human-readable labels for seniority, work model, role family.
app/page.tsx	Pulse dashboard cards and charts.
app/jobs/page.tsx	Job feed filters, export, table, compare link.
app/fit/page.tsx	Resume upload, selected job mode, fit cards.

Backend and data files

tracker.py	Batch orchestrator: load companies, scrape, sync, enrich, extract skills.
scraper/*.py	ATS-specific job fetchers returning normalized `JobPosting` objects.
scraper/enrichment.py	Infers seniority, work model, department, role family, location fields.
analysis/nlp.py	Extracts skill tags from job descriptions.
analysis/resume_fit.py	Resume parsing, shortlist scoring, Groq prompts, fit normalization.
db/models.py	SQLite connection, schema creation, migrations.
db/queries.py	All SQL reads/writes used by scrapers, API, dashboard, and fit matching.
api/app.py	FastAPI route layer and error sanitization.

Create the shell: theme, layout, sidebar

Global theme

```
:root {
  --background: #0f1117;
  --foreground: #e5e7eb;
  --card: #1a1d27;
  --card-border: #2a2d3a;
  --accent: #6366f1;
  --accent-light: #818cf8;
  --green: #22c55e;
  --red: #ef4444;
  --muted: #9ca3af;
}
```

Root layout

```
<body className="h-full flex">
  <Sidebar />
  <main className="flex-1 overflow-y-auto p-4 pt-16 md:p-8">
    {children}
  </main>
</body>
```

Sidebar logic

```
const NAV = [
  { href: "/", label: "Pulse", icon: Activity },
  { href: "/changes", label: "Changes", icon: ArrowLeftRight },
  { href: "/trends", label: "Trends", icon: TrendingUp },
  { href: "/companies", label: "Companies", icon: Building2 },
  { href: "/fit", label: "Resume Fit", icon: Sparkles },
  { href: "/jobs", label: "Job Feed", icon: Briefcase },
];

const active = pathname === href;
className={active
  ? "bg-accent/15 text-accent-light font-medium"
  : "text-muted hover:text-foreground hover:bg-white/5"
}
```

This is why every screenshot has the same frame: dark body, fixed sidebar, active nav color, and scrollable page content.

Create the database and query layer

Core tables to create first

companies: identity, ATS source, sector, company type, funding metadata.

job_postings: title, department, location, URL, role fields, status, active flag.

scrape_runs: run status, counts, errors, batch id.

job_events: added/removed events for Changes.

job_skills: many-to-many skill tags.

resume_profiles and **resume_job_fits:** AI matching cache.

Query functions by feature

get_company_stats	Companies table and coverage views.
get_active_jobs	Job Feed filters and company detail listings.
get_active_job	Selected job card on Resume Fit.
get_jobs_for_fit_pool	Candidate pool before deterministic scoring and Groq.
get_overview_stats	Pulse KPI cards.
get_recent_events	Changes event feed.
get_fastest_movers	Companies with the largest add/remove movement.
get_dept_sector_cross	Trend cross-tab charts.

Minimal query shape to copy

```
def get_active_jobs(company_type=None, search=None, skill=None):
    sql = """
        SELECT jp.*, c.name AS company_name, c.sector
        FROM job_postings jp
        JOIN companies c ON c.id = jp.company_id
        WHERE jp.is_active = 1
    """

    params = []
    if company_type:
        sql += " AND c.company_type = ?"
        params.append(company_type)
    return [dict(r) for r in conn.execute(sql, params).fetchall()]
```

Expose query functions with FastAPI

Route groups

<code>/api/overview</code>	<code>`overview` -> `get_overview_stats`</code>
<code>/api/companies</code>	<code>`companies` -> `get_company_stats`</code>
<code>/api/companies/{id}</code>	<code>`company_detail` -> detail, skills, departments</code>
<code>/api/jobs</code>	<code>`jobs` -> `get_active_jobs` with filters</code>
<code>/api/jobs/{job_id}</code>	<code>`job_detail` -> selected fit preview</code>
<code>/api/changes/events</code>	<code>`change_events` -> event feed</code>
<code>/api/fit/matches</code>	<code>`fit_matches` -> upload resume and search many jobs</code>
<code>/api/fit/jobs/{job_id}</code>	<code>`fit_job` -> upload resume and compare one job</code>

Normal route pattern

```
@app.get("/api/jobs")
def jobs(company_type: Optional[str] = None,
         search: Optional[str] = None,
         skill: Optional[str] = None):
    return queries.get_active_jobs(
        company_type=company_type,
        search=search,
        skill=skill,
    )
```

Error translation

The backend catches low-level Groq/config failures and returns product-safe messages. That keeps deployment details out of the user interface.

Create a typed frontend API client

Why this matters

Without `api.ts`, every page would hand-roll URLs, form uploads, error parsing, and response types. A single API client turns backend routes into reusable frontend functions.

- 1 Define TypeScript interfaces matching backend response shapes.
- 2 Write `fetchApi<T>` for GET routes with optional query params.
- 3 Write `postFormApi<T>` for file uploads and parsed backend errors.
- 4 Export named API functions like `jobs`, `companyDetail`, `fitMatches`, and `fitJob`.

Types drive safer UI code

```
export interface JobRow {
  id: number;
  company_id: number;
  title: string;
  company_name: string;
  sector: string;
  seniority: string | null;
  work_model: string | null;
  role_family: string | null;
  url: string | null;
}

jobs: (filters?: JobFilters) =>
  fetchApi<JobRow[]>("/api/jobs", filters)

jobDetail: (id: number) =>
  fetchApi<JobRow>(`/api/jobs/${id}`)
```

Build each visual page from the same recipe

Reusable page recipe

- 1. Import:** React hooks, API helpers, formatters, chart components.
- 2. State:** create arrays for API rows and controls for filters/toggles.
- 3. Fetch:** call API helpers inside `useEffect`; include control values in dependencies.
- 4. Derive:** use `useMemo` for filtered lists, cross-tab transforms, active filter counts.
- 5. Render:** header, controls, cards/charts/table, empty/loading state.
- 6. Link:** use `next/link` for navigation between related screens.

Generic page skeleton

```
export default function SomePage() {
  const [rows, setRows] = useState<RowType[]>([]);
  const [filter, setFilter] = useState("");

  useEffect(() => {
    api.someEndpoint(filter || undefined).then(setRows);
  }, [filter]);

  const visibleRows = useMemo(() => {
    return rows.slice(0, 500);
  }, [rows]);

  return (
    <div className="space-y-6">
      <Header />
      <Controls value={filter} onChange={setFilter} />
      <Visual rows={visibleRows} />
    </div>
  );
}
```

Build the resume AI feature carefully

Backend AI functions

<code>extract_resume_text</code>	Accept <code>.txt</code> , <code>.md</code> , <code>.pdf</code> , <code>.docx</code> ; convert bytes to text.
<code>parse_resume_profile</code>	Ask Groq for strict JSON: roles, skills, seniority, domains, remote preference.
<code>_candidate_filters</code>	Convert profile into DB filter terms.
<code>shortlist_jobs</code>	Score DB rows deterministically before LLM evaluation.
<code>_request_fit_batch</code>	Ask Groq for fit score, verdict, why, gaps, pointers.
<code>_normalize_fit</code>	Guarantee the UI receives safe fields even if LLM output is incomplete.
<code>save_resume_fit</code>	Cache result by resume id, job id, prompt version.

Frontend AI functions/components

<code>FitInner</code>	Owns mode, form, submit, results, query param sync.
<code>handleSubmit</code>	Chooses <code>/api/fit/matches</code> or <code>/api/fit/jobs/{id}</code> .
<code>SelectedJobCard</code>	Makes URL-selected jobs obvious to the user.
<code>ProfileRow</code>	Displays parsed resume profile facts.
<code>FitCard</code>	Displays per-job score, matched skills, explanation blocks.
<code>ReasonBlock</code>	Reusable list renderer for why, gaps, and pointers.

Safety constraints to preserve

Do not store raw resume text. Keep `GROQ_API_KEY` only on the backend. Use low temperature and JSON response format. Cache structured profile and fit results, not the uploaded file.

Build the scraper and enrichment pipeline

Scraper function contract

```
async def parse(company_name: str, ats_identifier: str) ->
list[JobPosting]:
    # Fetch provider-specific jobs
    # Normalize each result into JobPosting
    # Return title, department, location, url, external_id,
description
    return jobs
```

Every ATS adapter hides provider-specific API or HTML details behind the same `parse` shape.

Orchestrator flow

```
companies = load_companies()
for c in companies:
    cid = queries.upsert_company(c)

results = await asyncio.gather(*[
    scrape_company_limited(semaphore, c, company_ids[c["name"]])
    for c in companies
])

enrich_all()
extract_all_skills()
```

Sync jobs

New jobs become active rows. Missing jobs are marked inactive and produce removal events.

Enrich jobs

Titles and locations become normalized fields used by filters and charts.

Extract skills

Descriptions become `job\_skills`, powering skill filters and deterministic resume matching.

Build order for a nontechnical beginner

Part A: make it run

1. Create a repo with two folders: backend at root and frontend in `dashboard/`.
2. Install backend packages: FastAPI, Uvicorn, python-multipart, pypdf, python-docx.
3. Create a Next.js app in `dashboard/` with TypeScript and Tailwind.
4. Add `db/models.py` with SQLite connection and table creation.
5. Add `db/queries.py` with one query at a time, starting with overview and active jobs.
6. Add `api/app.py` and expose `/api/overview` and `/api/jobs` first.
7. Add `dashboard/src/lib/api.ts` and verify the frontend can fetch backend JSON.

Part B: make it useful

8. Build the root layout, global theme, and sidebar navigation.
9. Build the Pulse page with KPI cards and one chart.
10. Build the Job Feed with search, company type, skill filters, and table rows.
11. Build Companies, Changes, Trends, Skills, Roles, Coverage, and Health using the same fetch/render pattern.
12. Add scraper adapters and run the local tracker until the DB has enough data.
13. Add enrichment and skill extraction so filters and charts become meaningful.
14. Add Resume Fit after the normal job data experience works.

Exact feature order to recreate the UI

1. Navigation shell

Create ``Sidebar``, ``RootLayout``, and design tokens. You should be able to click between empty pages before adding any data.

2. Typed data client

Create ``fetchApi``, ``postFormApi``, and TypeScript interfaces. Replace raw fetch calls with ``api.overview()``, ``api.jobs()``, and `friends`.

3. Dashboard primitives

Build ``KpiCard``, shared tooltip style, chart card shells, and formatters for enums.

4. Search screens

Use controlled inputs, ``useMemo`` filter objects, and ``useEffect`` fetches. Render tables from arrays.

5. Chart screens

Use query helpers to return rows already shaped for charts, then pass arrays to `Recharts`.

6. AI screens

Start with file upload and selected job preview, then add backend parsing, shortlist scoring, Groq evaluation, and result cards.

Definition of done

A beginner can recreate this app when they can explain each screen as: state variables, API call, returned data type, visual component, and user action that updates state or navigates.

Function map by layer

Frontend

`Sidebar`, `KpiCard`, `PulsePage`, `JobFeedPage`,
`FitPage`, `FitInner`, `SelectedJobCard`, `ProfileRow`,
`FitCard`, `ReasonBlock`, `statusBadge`,
`getStatusKey`, `CompanyCard`, `CompanyListTable`,
`sortSeniority`, `mapWorkModelLabels`, plus API
methods in `api.ts`.

Backend API

`overview`, `companies`, `company\_velocity`,
`company\_detail`, `sectors`, `departments`, `seniority`,
`work\_models`, `role\_families`, `job\_freshness`, `skills`,
`jobs`, `job\_filters`, `job\_trends`, `job\_detail`,
`change\_events`, `fastest\_movers`, `sector\_delta`,
`scraper\_health`, `\_fit\_error`, `fit\_matches`, `fit\_job`.

AI and data

`extract\_resume\_text`, `analyze\_resume\_matches`,
`analyze\_single\_job`, `parse\_resume\_profile`,
`shortlist\_jobs`, `\_candidate\_filters`, `\_evaluate\_jobs`,
`\_request\_fit\_batch`, `\_score\_job`, `\_chat\_json`,
`\_normalize\_fit`, `\_merge\_fit`, `init\_db`,
`get\_connection`, `sync\_jobs`, `get\_active\_jobs`,
`get\_jobs\_for\_fit\_pool`, `save\_resume\_fit`.

How to read the codebase

Pick one visible thing in a screenshot, find the component that renders it, find the API helper it calls, find the FastAPI route with the same path, then find the query function it calls. That trace is the app.